

VITYarthi Flipped Course

PROJECT REPORT

Student Performance & Attendance Analyzer

A Python-based command-line application for student data management, academic analysis, attendance tracking, and reporting.

Student Name	Akriti Mritunjay Singh
Registration Number	26BCE11022
Course	Python Programming
Project Type	Command-Line Application
Data Storage	JSON
Testing	pytest
Version Control	Git / GitHub
Institute	VIT Bhopal University

Academic Project Report

1. Introduction

Student Performance & Attendance Analyzer is a modular Python command-line application developed for the Python Programming course. The system provides a structured way to store student information, enter academic marks, record attendance, calculate grades and percentages, analyze overall performance, and generate reports.

The application uses JSON as a lightweight persistent storage mechanism and demonstrates Python functions, conditions, loops, exception handling, file handling, modular programming, data validation, and automated testing.

Project Goal: To create a simple, maintainable and testable academic data management system.

2. Problem Statement

Managing student academic information manually can be time-consuming and can make it difficult to keep marks and attendance organized in one place. The proposed solution is a command-line Python application that stores student data in JSON, performs calculations automatically, validates input, and presents results through menus and reports.

Target Users: Students, teachers/faculty members, and academic administrators.

3. Functional Requirements

The project provides five major functional modules: Student Management, Marks Management, Attendance Management, Performance Analysis, and Reports.

Module	Main Functions
Student Management	Add, view, search, delete students
Marks Management	Enter/view marks, total, percentage, grade
Attendance Management	Enter/view classes and attendance
Performance Analysis	Combine marks and attendance
Reports	Generate consolidated report

4. Non-functional Requirements

Usability: Menu-driven interface with clear prompts.

Reliability: Persistent JSON storage and controlled missing-record handling.

Maintainability: Separate Python files for major responsibilities.

Error Handling: Invalid numeric input and invalid ranges are checked.

Resource Efficiency: Lightweight local JSON storage.

Testability: Automated pytest tests.

5. System Architecture

main.py acts as the controller. student.py manages student records; marks.py manages marks and grades; attendance.py manages attendance; analyzer.py performs performance analysis; reports.py generates reports; validators.py checks input; students.json provides persistent storage.

6. Design Diagrams

Use Case: Primary operations include managing students, marks, attendance, performance analysis, and reports for student/teacher users.

Workflow: Start → Main Menu → Choose Module → Enter/Retrieve Data → Validate → Calculate/Analyze → Display/Report → Return or Exit.

Sequence: User → main.py → selected module → students.json → result returned to user.

Storage Design: JSON records contain id, name, course, marks, total, percentage, grade, and attendance.

7. Design Decisions & Rationale

Python was selected because the project is for Python Programming. JSON provides simple local persistence. Separate modules improve maintainability. A command-line interface keeps the focus on programming logic. pytest provides repeatable automated tests.

8. Implementation Details

Project files: main.py, student.py, marks.py, attendance.py, analyzer.py, reports.py, validators.py, requirements.txt, README.md, statement.md, data/students.json, tests/, and docs/.

Calculations: $\text{Percentage} = \text{Total Marks} / \text{Number of Subjects}$. $\text{Attendance Percentage} = (\text{Attended Classes} / \text{Total Classes}) \times 100$.

Grade boundaries: $A+ \geq 90$, $A \geq 80$, $B \geq 70$, $C \geq 60$, $D \geq 50$, otherwise F.

9. Screenshots / Results

Representative results from testing include successful student creation, marks calculation, attendance calculation, performance analysis, report generation, rejection of marks above 100, rejection of attendance exceeding total classes, and safe handling of non-numeric input.

Example: 18 attended classes out of 20 produces 90% attendance.

10. Testing Approach

Automated tests cover student data loading and structure, grade calculation, and attendance validation. The recorded test result is **8 automated tests passed** using pytest.

11. Challenges Faced

Designing modular responsibilities, connecting modules through main.py, handling invalid input safely, keeping JSON data consistent, configuring pytest imports, and preparing GitHub documentation.

12. Learnings & Key Takeaways

Practical Python programming, JSON file handling, modular design, input validation, exception handling, automated testing, basic software architecture, workflow documentation, and Git/GitHub usage.

13. Future Enhancements

Graphical user interface, more subjects and configurable grading, PDF/CSV export, SQLite database, charts, authentication, and advanced search/filtering.

14. References

VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
Python Documentation.
pytest Documentation.

Git Documentation.

15. Project Submission Checklist

Problem statement and objectives — prepared.
Functional modules — implemented.
Non-functional requirements — documented.
Architecture/workflow/UML-style design — prepared.
Modular source code — implemented.
Validation and error handling — implemented.
Automated testing — 8 tests passed.
README.md and statement.md — prepared.
requirements.txt — prepared.
Git version control — setup in progress.
Public GitHub repository — to be completed before portal submission.
Final terminal screenshots — recommended before final submission.

Student: Akriti Mritunjay Singh **Registration Number:** 26BCE11022